

Decoding Lecture 1 - EECE 571-Q

Kjell Moller, Arman Lotfalikhani

July 2026

1 Introduction

Current quantum hardware is noisy and susceptible to a wide range of physical errors that serve as a barrier for quantum computation. Because of this problem, Quantum Error Correction (QEC) is a primary focus of modern research, with the goal of developing a Fault-Tolerant quantum computer. However to achieve truly fault tolerant computation we need to find methods of suppressing physical error rate during Circuit-Level noise [3] to a point where computation is reliable. To overcome physical noise, we encode physical qubits as logical codes. Where we can then stabilize our states to be tolerant to various sets of errors. However when an error does occur, we need a method of tracing back to and applying a recovery operation to ensure that our logical state is unaffected by errors on its physical qubits.

The motivation behind this lecture is exploring the mathematic objective to this problem and the solutions currently devised to solve it. We will walk through how we can map a measured error syndrome into a recovery operation for our logical state, as well as how a decoder needs to fit into the overall QEC pipeline and the constraints it must operate within.

1.1 Background

Given a quantum code C , we can stabilize the code space with the stabilizer of the code S . Since all stabilizers commute with each other, and leave the code state unchanged, if an error is introduced we can identify the syndrome as $He = s(mod 2)$. The syndrome is representative of which stabilizers anticommute with the error, allowing us to identify which nodes. However, multiple physical errors may have the same syndrome in a degenerate code.

With a given stabilizer S , if a Pauli error is contained in the normalizer but not in the stabilizer, this will cause a logical error on the state as it commutes with all of the stabilizers but will induce a logical change:

$$e \in N(S) \setminus S$$

If an error is contained in the stabilizer:

$$e \in S$$

It will produce no syndrome but will leave the code state unchanged. Finally if

$$e \notin N(S)$$

it will produce some syndrome that we need to identify a way to correct and recover from. This is a core mechanic of fault tolerance, where faults cannot spread into more errors than the code can correct. Knowing all of this, we need some method of identifying and recovering from errors that are not stabilized in the code.

1.2 Learning Outcomes

Up to this point in the course we have effectively learned the theory behind QEC and how to apply QEC codes to stabilize our logical states and lower our logical error rate. These learning outcomes aim to extend the students knowledge gained in the course by demonstrating how we can map our measured syndrome to an applied recovery operation. We provide students with numerous practical use cases and algorithms which were previously alluded to, so they can effectively implement a decoder and finalize how to build a QEC circuit from start to finish.

By the end of the lecture, students should be able to effectively:

- Formulate quantum decoding as an optimization problem over recovery operations

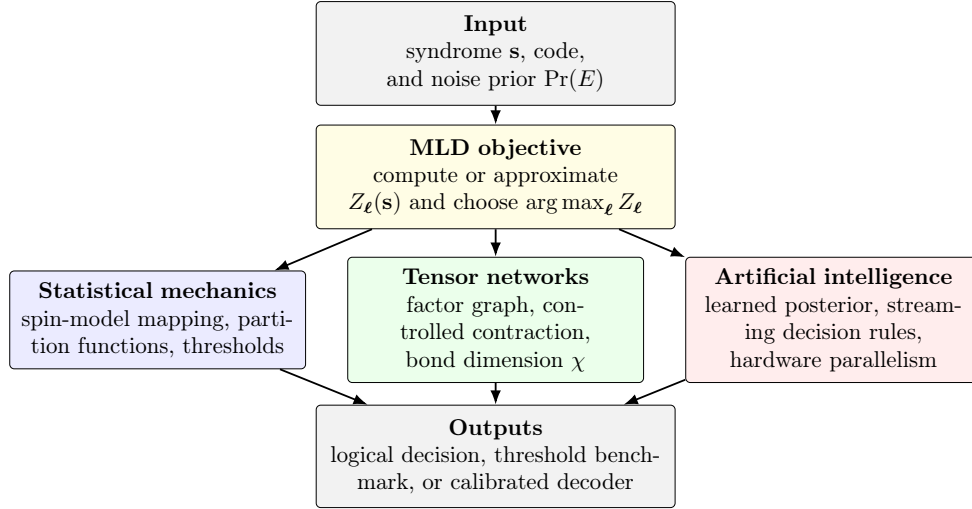


Figure 1: Classification of recent approaches to decoding. From: Figure 2 [3]

- Explain why maximum-likelihood decoding (MLD) is generally impractical and approximate decoders are needed
- Apply the algorithms of Belief Propagation (BP) and its variants, Minimum Weight Perfect Matching (MWPM) and Union-Find (UF), from syndrome to recovery
- Identify common performance metrics in QEC decoding, comparing algorithms and adapting to face the challenge of real-time decoding

2 The Decoding Objective

2.1 Maximum Likelihood Decoding

Let s be our observed syndrome on a parity check matrix H . Where $He = s$ defines our stabilizer measurement. The goal of a decoder is to map an observed syndrome to a recovery operation $s \rightarrow R$. Each decoders objective is to apply a recovery to ensure that when combined with our observed syndrome our logical state is changed. The decoding objective can be defined as [3]:

$$L^* = \underset{L}{\operatorname{argmax}} \{Pr(L|s)\}$$

Where L is the logical class. In reality this problem is intractable. As we scale up the distance of our codes we end up summing over an exponential amount of states consistent with our observed syndrome. Which is not a feasible problem to be solved in practice.

2.2 Decoder Families

Given that MLD cannot be solved efficiently, some method needs to be devised to approximate the objective function. This is where surrogate functions come into play.

Figure 1 proposes 3 (4) different families of decoders that encompass current research. Statistical mechanics, tensor network, and artificial intelligence based decoders (along with classical decoders). Each one of these families, and decoders within them will have some other mathematical objective function that approximates the decoding objective function. Often, classical algorithms are implemented as functional baselines for research on other decoder families, but can perform well on their own.

2.3 Tanner Graphs and Geometrical Properties

The factor graph of an $[n, k, d]$ classical code is a bipartite graph $G = (V, U, \Lambda)$ with an adjacency matrix given by the code's parity check matrix H . [26]

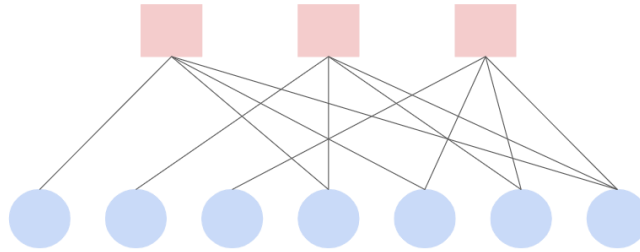


Figure 2: Tanner Graph of the $[n,k,d] = [7,4,3]$ Hamming Code. With check nodes in red and variable nodes in blue.

Tanner graphs allow us to easily visualize our codes stabilizer and qubit relationships by defining our data nodes or variable nodes as the columns of our parity check matrix H , and stabilizers or check nodes as our rows of the matrix. A graph edge is drawn between each node where the corresponding row and column values $H_{i,j} = 1$.

With our visualization, we are able to easily identify how errors can propagate through our graph and . The density of edges, formally known as the sparsity directly impacts how a given syndrome applied to a check node can impact many data qubits at once.

An (lQ, qQ) -QLDPC code family is defined as a set of CSS codes whose quantum parity check matrices H_{CSS} have row and column weights upper bounded by lQ and qQ respectively. [6, 22]

QLDPC codes are a primary focus of modern QEC as they prevent the edge weight from increasing to a level that hinders computation. Surface codes, Hamming codes, and the many of the other codes we have seen in class fall under this umbrella.

Since our edge weight is bounded on QLDPC codes, their Tanner graphs have geometrical properties that can be exploited. As we will see in the coming sections, knowing that there is a maximum edge weight can lead to limitations on how errors can appear on our check nodes, allowing us to find surrogate functions for our decoding objective and find better time complexity for decoding.

3 Classical Decoding

This section explores conventional decoders that are used in current practice as baselines for modern decoder architectures.

3.1 Union-Find

For topological codes, errors exhibit a pairlike structure in their tanner graphs. As they are CSS codes, we can isolate by X and Z type stabilizers to obtain a tanner graph where each variable node connects to two check nodes of each stabilizer type. To set up Union Find on an unrotated surface code, first the graph is transformed into a similar structure where check nodes are represented by a node on the graph, and variable nodes as edges between. We then define a boundary on the left and right edges of the graph where the outermost edges connect to. Quiz Question 2 in Figure 9. (a) visually outlines the setup of UF.

The algorithm begins with identified syndrome defects on check nodes, where a cluster $C_i = v_s$ is defined at each node with a syndrome defect. The algorithm then expands each cluster by half an edge each step, according to the following rules [7, 33]:

1. If two clusters touch, merge the two clusters into one larger cluster
2. If the parity of a cluster's syndrome defects is even, terminate the cluster's growth
3. If a cluster touches a border, terminate the cluster's growth
4. If the parity of a cluster's syndrome is odd, continue growing in the next step

A good video that was drawn as inspiration for quiz questions [31] illustrates the growth of these clusters as well as the relation to the following algorithm covered, Minimum Weight Perfect Matching (MWPM). Once all clusters have terminated their growth, each syndrome defect should be paired within each cluster. To accomplish this, we

apply the peeling algorithm [9]. This algorithm allows us to find an error chain connecting each defect. We first initialize a spanning forest between each disconnected cluster of its vertices and edges:

$$F_\epsilon = (V_F, E_F)$$

We then apply the identified syndrome to check nodes, where $s(v) = 1$ if an error is identified and $s(v) = 0$ otherwise. We then check each leaf of F_ϵ and apply the following rules:

1. If $s(v) = 1$: we include the edge between vertex v and its parent u $e = (u, v)$ in the error chain ϵ_c
2. If $s(v) = 0$: we do not include V in the error chain ϵ_c
3. Remove the leaf V from F_ϵ

This process terminates once there are no more vertices in the spanning tree. After which we should have some matching path between each syndrome defect. One intuitive video explaining the algorithm is [28]. Peeling will always produce a valid matching between syndrome defects within clusters, but it will not necessarily always be minimum weight [19]. This motivates the next algorithm that we will cover.

3.2 Minimum Weight Perfect Matching

Union-Find's issue with finding a minimum weight matching is that it does not preserve internal cluster information. Any defect can be paired with any other defect within a cluster so long as it produces a valid, not necessarily optimal, recovery. [15] The Blossom Algorithm is a method devised to retain this internal cluster information. MWPM implements the blossom algorithm's dual variable tracking method for cluster sizes [33]. For our graph defined as above with UF with variable nodes as edges E and check nodes as vertices V . The edge weight w_e is defined as the cost of the solution error path. The primal objective function is defined as:

$$\min_x \sum_{e \in E} w_e x_e$$

Subject to the constraint of

$$\sum_{e \in v} x_e = 1$$

This constraint implies that each vertex with a defect is paired with only one edge part of an error chain. And the objective allows us to minimize the distance of the error chain. The dual objective is defined as:

$$\max_y \sum_{s \in O} y_s$$

Subject to the constraint of

$$\sum_{s \in O} y_s \leq w_e$$

Intuitively this is the amount that each region has grown from it's origin. The edge slack can be defined as:

$$slack(e) = w_e - (y_u + y_v)$$

A tight edge occurs when $slack(e) = 0$, which occurs when the blossoms of each cluster touch.

With these definitions in mind, the rules for MWPM state:

1. Grow our dual variables y , which define the growth of each region, outward at each step
2. If slack becomes zero between two dual variables. We consider the following options
3. If the dual cluster being touched is unmatched, we match the two defects and terminate growth
4. If the dual cluster being touched already has a valid matching (even parity). We engage in alternating growth between the matched cluster to allow the existing cluster to continue growth through the existing matching. This effectively means that the closer dual variable to the collision will shrink, and another will grow to permit the other cluster to grow through (see [31]). If alternating growth does not find another matching, another blossom will form around all clusters

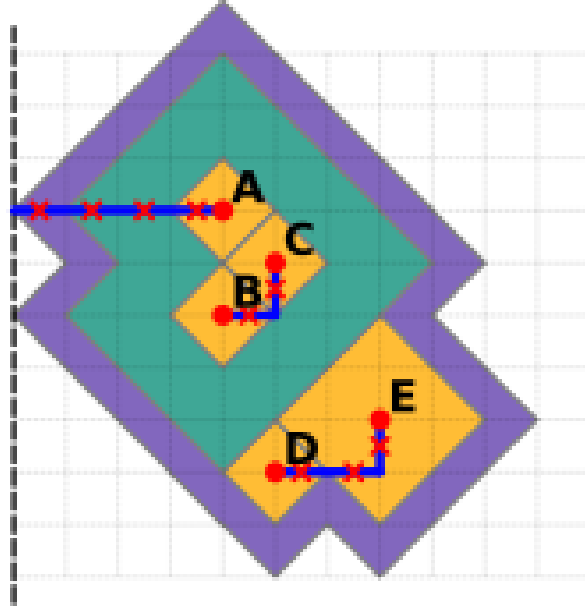


Figure 3: Figure 7, (8) from [33], illustrating the blossom algorithm on a surface code syndrome

5. If the dual cluster being touched forms an odd group of clusters, we fuse these into a larger vertex and begin a blossom, which is a new dual variable search region containing the growth of all of the clusters
6. We then grow each blossom outwards until all clusters/blossoms terminate with pair matchings minimizing the edge weight and maximizing our dual variables

This algorithm ensures that defects can only be matched with other defect contained within the same blossom. Circumventing the problem that UF had with a lack of internal cluster information. The annotated lecture slides contain an illustration of the blossom algorithm. To accompany this Figure 7 of [33] has a strong visualization of the different steps in the algorithm.

When comparing the two algorithms, Union-Find performs the same as MWPM under conditions of cluster touching both boundaries, and UF clusters having the same shape as MWPM clusters in one solution [31]. Some current improvements over the original blossom algorithm include sparse blossom [19], and fusion blossom [32], which parallelizes the blossom algorithm to achieve better performance.

3.3 Belief Propagation

While MWPM and Union-Find function well for the pair like structure of surface codes, they do not extend well to codes without this structure. For a more universal decoding algorithm, Belief Propagation extends much better to a broader family of codes.

Belief propagation functions by going through a series of cycles on the Tanner graph, where based on the syndrome it will update the probability or "beliefs" that certain variable nodes had an error occur. At each iteration it will check if any nodes have a belief greater than 0.5, and make a hard decision of whether that node had an error. BP follows the following sequence of operations:

1. Initialize each variable node with the physical error probability p_e
2. Send messages from variable to check nodes, where $\mathcal{N}(c) \setminus q$ is all edges connected to c besides q , defined as (Equation 9 [20]):

$$m_{q \rightarrow c}(E_q) \propto p(E_q) \prod_{c' \in \mathcal{N}(q) \setminus c} m_{c' \rightarrow q}(E_q)$$

Meaning our message is proportional to taking a product of all of our prior messages that did not come from the check node being sent to, and multiply it by our current belief

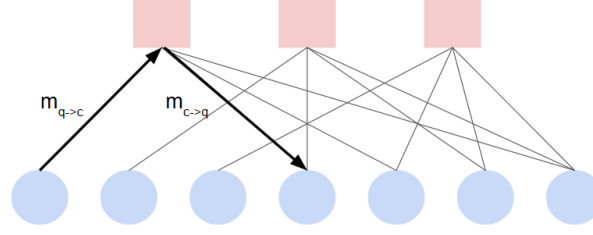


Figure 4: Belief Propagation Message transfer on the Hamming Code

3. Return messages from check to variable nodes given their new probabilities, defined as (Equation 8 [20]):

$$m_{c \rightarrow q}(E_q) \propto \sum_{\substack{E_{q'} \\ q' \in \mathcal{N}(c) \setminus q}} (\delta_{S_c, S_c \cdot E_c} \prod_{q' \in \mathcal{N}(c) \setminus q} m_{q' \rightarrow c}(E_{q'}))$$

Meaning our message is proportional to summing all of our products of prior messages from variable to check nodes multiplied by the current syndrome

4. Update the probabilities of each variable node given the new information (Equation 10 [20]):

$$p_q(E_q) = p_{q \text{ prior}}(E_q) \prod_{c \in \mathcal{N}(q)} m_{c \rightarrow q}(E_q)$$

Here the new belief probability is the prior belief multiplied by the product of all other check to variable messages

5. Check each variable node i and make a hard decision on whether $0.5 \leq p_i$
6. If a hard decision was made on $\hat{e}$, double check whether $H\hat{e} = s$. If so, terminate. If not, continue iterations

However, Belief Propagation has a few fundamental flaws, which are the short cycle and degeneracy. A short cycle is a series of connected edges in the Tanner graph where through following a series of edges you will trace back to the same node. Short cycles and degeneracy can cause information to spin in BP and as a result will make it converge much slower or not at all. As such, BP needs some other algorithm or decision to overcome this.

3.3.1 Ordered Statistics Decoding

To circumvent these issues with BP, one option is to use Ordered Statistics Decoding (OSD). OSD produces a syndrome consistent correction if the hard decision of BP does not match the syndrome. Once BP is finalized, it provides us with the probability of each variable node having an error. Using these, the steps of OSD-0 are as follows [22]:

1. Reorder the columns of H in order of each columns BP error probability, ordered from the highest values on the left to lowest on the right of the new matrix to obtain H_{OBP}
2. Partition the new H_{OBP} into a linearly independent set of $\text{rank}(H)$ variables as a set S , and the remainder set T
3. Solve for S where

$$e_{[S]} = H_{[S]}^{-1} \cdot s$$

. The inverse matrix here indicates a solved matrix using Gaussian Elimination

4. Set the remaining columns to 0 and assume that the correction is contained in the most likely set S given by BP

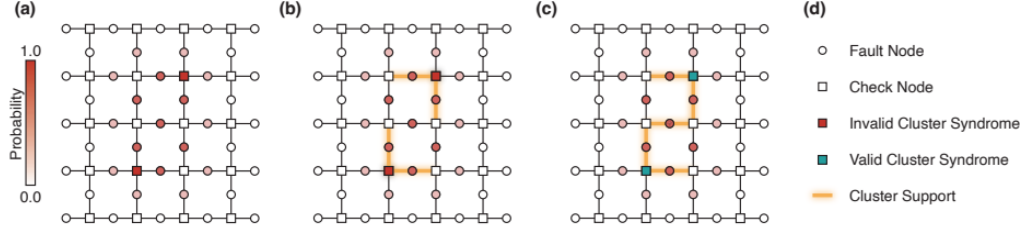


Figure 5: Figure 2. of [14] Localized statistics decoding reliability-based weighted cluster growth for the surface code

Higher order OSD consists of testing configurations not set to 0 for the dependent variable set T as we consider (Equation 8 [22]):

$$e_{[S,T]} = (H_{[S]}^{-1} \cdot e_{[S]} + H_{[S]}^{-1} \cdot H_{[T]} \cdot e_{[T]}, e_{[T]})$$

Where we sweep across possibilities of $e_{[T]}$ according to the weights.

In the case of OSD-0 this simplifies to

$$e_{[S,T]} = (e_{[S]}, e_{[T]}) = (e_{[S]}, 0)$$

OSD's main issue is that this matrix solve needs to run globally, meaning that at each decoding cycle with OSD there is a large computational overhead of the reliability sorting and matrix solve. Especially with larger codes, this can dominate the overall decoders time complexity, which is non ideal as we will discuss in section 4.

3.3.2 Localized Statistics Decoding

Since OSD suffers from the performance of having to do a global matrix solve on each decoding cycle, which does not scale well. [14] proposes a variant of OSD where instead of a global matrix solve we isolate to local clusters. This reduces the computation required significantly and also gives the ability to parallelize the algorithm through on the fly elimination on smaller clusters, in a similar way to fusion blossom [32].

Rather than solving the global decoding equation $He = s$ directly, we initialize a cluster C_i beginning at each measured syndrome defect. After BP completes, we expand the cluster to include a neighboring node with the highest BP probability. Each cluster parity check matrix H_{C_i} will then be expanded to include these new nodes, as visualized in Figure 5. *On the fly elimination* involves applying a Gaussian Elimination step at each cluster expansion, where we only need to eliminate the added column rather than an entire matrix solve to check if our local solve can satisfy the equation

$$H_{C_i} e_{C_i} = s_{C_i}$$

If this solve is not successful, continue adding the next most likely node on the clusters locality to find the our next addition. This repeats until we satisfy our syndrome equation. In the case of two clusters colliding, we merge their respective matrices and form one larger cluster that we will continue applying on the fly elimination to and expanding until we satisfy the combined syndrome equation. Once every cluster has completed we merge their corrections as a union to find our final solved $\hat{e}$.

By partitioning our clusters we are able to parallelize our computing resources to more efficiently solve for our syndrome on our code. Not just this, but it also limits the size of the matrix solve of each Gaussian Elimination step, as only in the worst case will our clusters expand to cover the entire code and effectively become an OSD solve.

4 Real-Time Decoding

4.1 The Backlog Problem

Practical decoder implementations are not just limited by their scalability and ability to suppress errors, but also by their runtime. [27] proposes a recursive backlog constraint, where if our decoder implementation is not able to keep up with the throughput of syndrome generation by the hardware, we will encounter an exponential growth in the backlog of syndromes waiting to be decoded. If we have a queuing time from our generated syndromes t_q and processing time of the decoder t_p , we can define the ratio between these as:

$$f = \frac{t_p}{t_q}$$

At each decoding cycle D_k we can define the time taken to decode if D_k depends on the measurement of D_{k-1} as:

$$D_k = f D_{k-1}$$

As we expand our number of decoding cycles n , we can see that if $f > 1$ we will see an exponential growth in decoding time of $D_n = f^n D_0$. In light of this, our decoder implementations need to have an as good or better throughput than our hardware to avoid this exponential queuing backlog.

Modern systems require an estimated syndrome decoding cycle time of approximately $1\mu s$ for superconducting transmon qubits and $1ms$ for trapped ion qubits [1]. This motivates research on not just finding strong decoders which can effectively minimize our logical error rate, but also aiming for these decoders to operate under the constraints described for the relevant hardware.

4.2 Detector Error Models

Detector Error Models (DEM) are an abstraction enabling efficient and methodical translation of our circuit level representations into a node representation easily usable by our decoders. They enable us to compile our Clifford circuits down to a matrix representation similar to a Tanner Graph of the code and stabilizers. This detector error matrix contains information on which errors violate which detectors, as well as the associated probability of each error occurring, and whether they will flip a logical observable on our code [10].

Within the matrix, each detector is formed by comparing stabilizer measurements between consecutive rounds:

$$d_1 = m_{1,t} \oplus m_{1,t-1}$$

This allows it to not just observe stabilizer measurements but be able to differentiate between rounds and decide whether there is a difference in the new measurement parity. Similarly we can then define our error matrix Ω where $\Omega_{i,j}$ contains our measurement outcomes in rows and fault mechanisms in columns (think of them as locations where errors may occur and propagate). The matrix is indexed with 1 where a column error would propagate to that row's measurement. Our DEM matrix then can be defined as

$$H = D\Omega$$

The DEM effectively functions as a bridge between our decoder and the circuit, as the decoder only cares about the measured syndrome associated with the code being employed.

$$Circuit \rightarrow DEM \rightarrow Decoder$$

This is very useful for simulation, allowing us to fire detectors with events and analyzing how our decoder responds to errors. We only need to compile the circuit once into the model, which is then able to be rapidly interpreted by the decoder.

4.3 Comparing Decoders

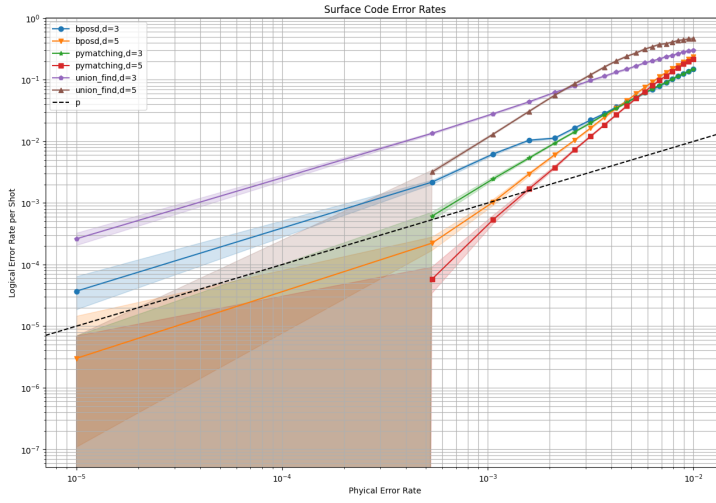
Different use cases will have different requirements around decoder implementations. One scenario may involve needing to be a live real time system, where although accuracy is ideally high the system must run quickly enough to avoid the *backlog problem*. This contrasts with a case where a circuit is being run in a simulation to test accuracy where runtime is not as imperative.

Some of the defining metrics used to compare decoders include:

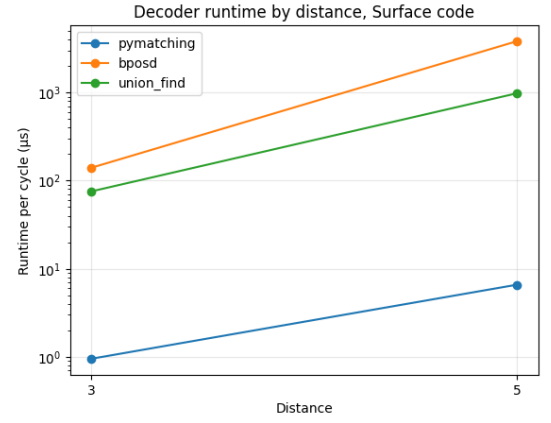
- Error threshold p_{th} , the physical error rate below which scaling our code distance will present a decrease in logical error rate
- Decoding runtime T_d per decoding cycle
- Logical error rate p_l , compared across distances

Testing these metrics in conjunction with analyzing the use case of the decoder will give insight into the best decoder for the situation. There is no outright "best" decoder, it is highly dependent on the geometry of the code in question and use case. This motivates the demonstration provided during the lecture, where various decoders were compared on rotated surface code and color code circuits to showcase differences in runtime and logical error rate suppression.

Of note is that [21]'s Open Source LSD implementation is not yet parallelized, and so is expected to exhibit similar or potentially slightly worse performance than OSD given the context switching required to handle multiple clusters.

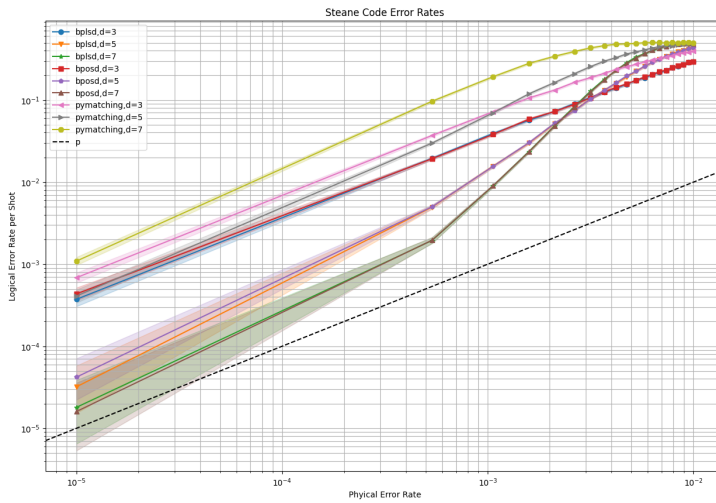


(a) Surface Code Error rate

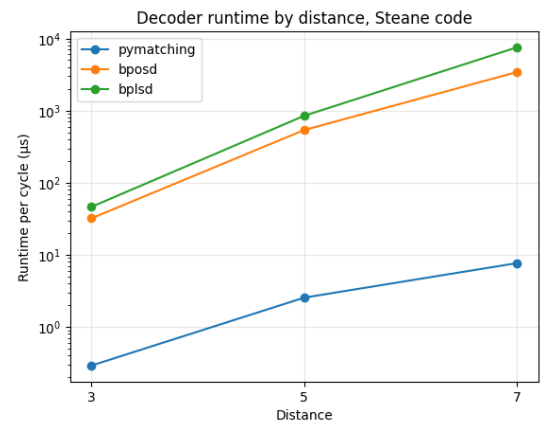


(b) Decoder runtime

Figure 6: Error rate and runtime results for the rotated surface code on Z stabilizers



(a) Steane Code Error rate



(b) Decoder runtime

Figure 7: Error rate and runtime results for the Steane code

5 Supporting Material

5.1 Demonstrations

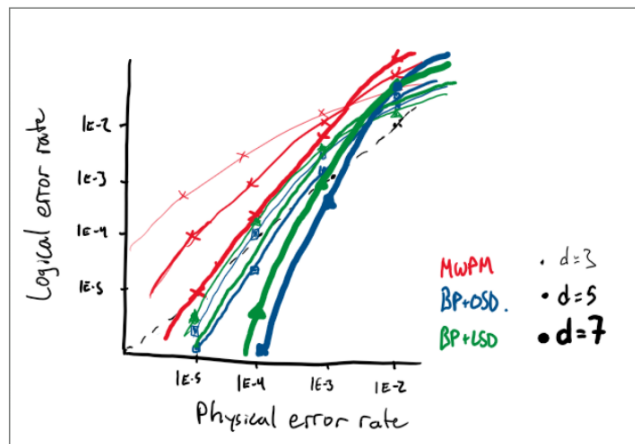
Here we present some figures from the demonstration presented in lecture. Decoders simulated include [21, 19] in the demonstrations. These demonstrations were simulated in Sinter using circuits built using Stim. Simulations were ran on a maximum of 1,000,000 shots and 10,000 errors per code distance on each code up to distances of 7. With greater computing resources this experiment could be extended to greater code distances.

Demo 1: Figures 6, 7 outline results from the first demonstration. This involved comparing various baseline decoders on multiple codes utilized in class using Stim and Sinter Clifford simulation. The goal of this demonstration was to show that decoders vary in performance across different codes, and also to illustrate how one can obtain results to compare decoders as described in section 4.3.

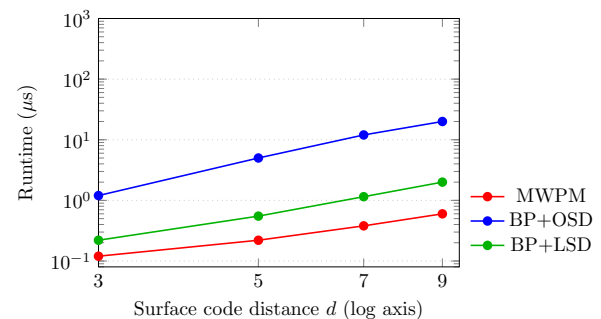
In the figures, most notable Pymatching performed best on the surface code by a significant margin, both in runtime and logical error rate suppression, with a much lower threshold value than BPOSD and UF. However, when transitioning to a Steane Code, the only codes that project as viable decoders at a higher distance are BPLSD and BPOSD, even though Pymatching shows a much better runtime. Once again the simulation was limited by computing resources, so at lower physical error rates the simulation could not be executed fully. Decoders must pass the $x = y$ barrier between the physical and logical error rate to be considered viable, as anything that introduces a higher logical than physical error rate is doing more harm than good. Many of the decoders in the demonstration do not cross this threshold, and so should not be considered as options for that specific code and experiment.

5.2 Quiz Questions

Quiz Question 1: a) An experiment requires a quantum decoder with strong logical error rate suppression. Runtime



(a) Physical vs. Logical Error rates for different decoders on the surface code.



(b) Decoder runtime

Figure 8: Quiz Question 1

matters but is not as important as accuracy and scalability since we are not running in real time. Which decoder is most appropriate?

Answer: BP+OSD Lowest error rate as well as shows most optimal trend in logical error rate scaling across distance.

b) A quantum device requires fast decoding that runs in real time on a trapped ion system. Accuracy is important but it needs to avoid an exponential growth in decoding time. Which decoder is most appropriate?

Answer: BP+LSD Both MWPM and BP+LSD are under the 1 microsecond threshold. So although MWPM is faster BP+LSD is a better choice since it does not have an exponential growth in decoding time and has higher accuracy.

Rationale: This question mainly targets learning objective 4 in lecture 1: Identify common performance metrics in QEC decoding, comparing algorithms and adapting to face the challenge of real-time decoding. Helping students make educated choices on the right decoder for the task while also testing the knowledge of RT decoding constraints

Quiz question 2: Syndrome defects are found at check nodes highlighted in red, with a corresponding number.

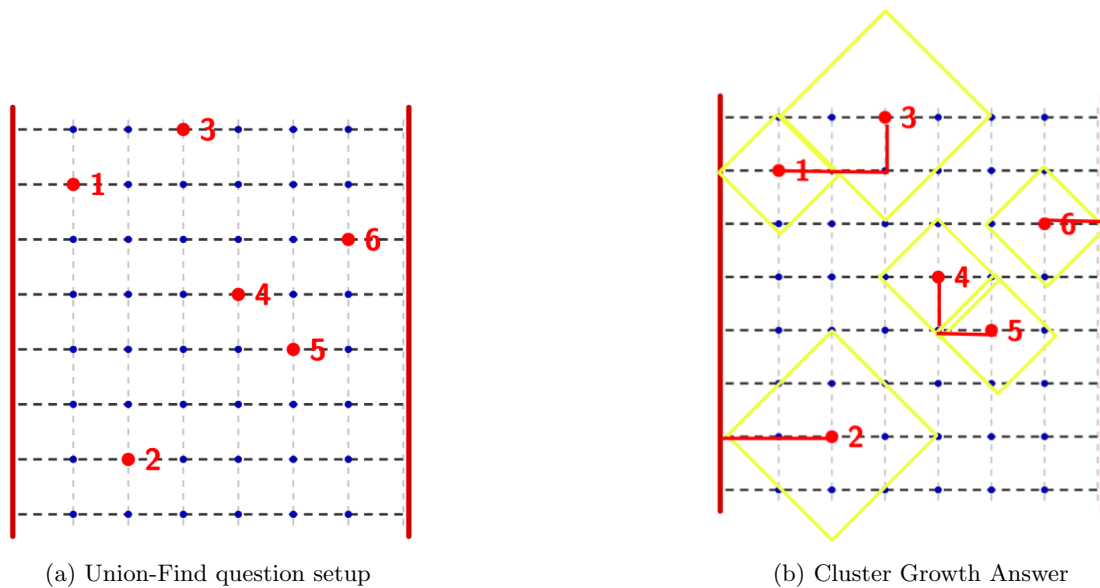


Figure 9: Quiz Question 2

Apply the Union find algorithm and pair each syndrome together with another. If paired with a boundary use a capitalized B rather than a number. Express the pairs separated by commas, ordered with no spaces (ie. 12,34,56,78,9B. Please order in ascending order of sums, with B being worth 10, so that prairielearn doesn't get confused with answers.

Answer: 13,45,2B,6B Cluster growth is represented in Figure 9. (b).

Rationale: Inspired by [31], this question mainly targets learning objective 3 in lecture 1: Apply the algorithms of Belief Propagation (BP) and its variants, Minimum Weight Perfect Matching (MWPM) and Union-Find (UF), from syndrome to recovery. As Union Find is objectively the easiest to trace through of the four introduced in class by hand.

6 Conclusion

After completing this lecture, students should have a solid grasp of how a measured syndrome is translated to a recovery operation. They should also have some practical strategies and algorithms that they can understand and implement in UF, MWPM, and BP and its variants. They should also be able to identify scenarios and codes where each algorithm makes conceptual sense to implement and compare decoder performances through simulations and experiments.

7 Learning outcomes of lecture 2 and rationale

By the end of the lecture, students should be able to:

- Describe the limitations of methods such as MWPM, and be able to justify correlated MWPM as a partial correction.
- Describe opportunities for machine learning to enter the quantum decoding pipeline
- Distinguish between ML decoder architectures such as CNNs, GNNs, and transformers
- Justify which ML decoder architecture should be chosen given the benefits and limitations of each method and the QEC code

The lecture notes for this section are organized as follows: First, we revisit the MWPM problem from the maximum log-likelihood perspective, which enables the discussion of how to handle correlations in MWPM. This

is motivated by the practical limitation that we are submitting individual lecture notes, and I planned to modify this part and clarify some aspects in the lecture 1 notes. As a result, I have only included my own additions and modifications here. The other rationale is that during my presentation, I briefly (and verbally) mentioned correlated MWPM as a baseline AlphaQubit compared itself to. We are using this chance to expand more on this topic, which in turn explains more of the limitations of MWPM in addition to enriching the structure. After this, we move on to a concise explanation of the concepts in tensor networks and how they could be utilized in decoding. After this, students should have a solid grasp of several non-machine-learning methods for decoding so far. In the next phase, we review some machine learning background to explain how a model architecture can be adjusted to take advantage of the code structure. The building blocks included are influenced by AlphaQubit 1 [2] and 2 [23], with the addition of graph neural network decoding. Neither this background nor the methods discussed are exhaustive, and many more approaches, such as reinforcement learning for decoders, are omitted to reduce the length and improve clarity.

8 Maximum likelihood decoding (review)

Probability of logical class given syndrome sequence $\mathbf{s}$:

$$\mathbb{P}(\ell | \mathbf{s}) = \frac{1}{\mathbb{P}(\mathbf{s})} \sum_{E \in C_\ell(\mathbf{s})} \mathbb{P}(E) \propto Z_\ell(\mathbf{s})$$

Choosing error with the highest probability of occurring:

$$\hat{\ell}_{\text{MLD}} = \arg \max_{\ell \in \{0,1\}^{2k}} Z_\ell(\mathbf{s}) \quad Z_\ell(\mathbf{s}) := \sum_{E \in C_\ell(\mathbf{s})} \mathbb{P}(E),$$

9 MWPM revisited

9.1 Minimum Weight Perfect Matching

First, consider the maximum likelihood detector, under the assumption that errors on each qubit happens independently. Similar to the UF case, map variable nodes (qubits) to edges E and check nodes (stabilizers) as vertices V . Under the independent noise assumption, the total probability of error E happening will be [15]

$$\begin{aligned} \mathbb{P}(E) &= \prod_{e_i \in E} \mathbb{P}(E_i) \prod_{e_i \notin E} (1 - \mathbb{P}(E_i)) = \left(\prod_{e_i \in E} \frac{\mathbb{P}(E_i)}{1 - \mathbb{P}(E_i)} \right) \prod_{\substack{e_i \notin E \\ e_i \in E}} (1 - \mathbb{P}(E_i)) \\ \log \mathbb{P}(E) &= \sum_{e_i \in E} \log \left(\frac{\mathbb{P}(E_i)}{1 - \mathbb{P}(E_i)} \right) + \log \prod_{\substack{e_i \notin E \\ e_i \in E}} (1 - \mathbb{P}(E_i)). \end{aligned}$$

The second term is a constant that does not depend on any specific error E ; also note that $\log \left(\frac{\mathbb{P}(E_i)}{1 - \mathbb{P}(E_i)} \right) \leq 0$ for $\mathbb{P}(E_i) \leq \frac{1}{2}$. If instead of finding the maximum likelihood logical class, we focus on the maximum likelihood single error sequence, our objective becomes

$$\max \sum_{e_i \in E} \log \left(\frac{\mathbb{P}(E_i)}{1 - \mathbb{P}(E_i)} \right) = \min \sum_{e_i \in E} -\log \left(\frac{1 - \mathbb{P}(E_i)}{\mathbb{P}(E_i)} \right),$$

subject to satisfying the syndrome sequence constraints. So far we have concerned ourselves with the decoding graph. To create a syndrome graph, we only gather the non-trivial syndrome nodes, and create a weighted fully connected graph. If we assume all $\mathbb{P}(E_i)$ to be equal, each edge weight will be the Manhattan distance (sum of vertical and horizontal distance on a surface code) between syndromes [33]. Then, the syndrome conditions would turn into the perfect matching condition in MWPM. To formalize the problem, we first need to define appropriate notation, following the convention of [11] and [19]. For every vertex $v \in V$, consider $\delta(v)$ as its set of neighbors $\delta(v) = \{u \mid (v, u) \in E\}$. For a nontrivial (cardinality more than 1) subset of vertices $S \subseteq V$, we extend the neighboring definition to $\delta(S) := \{(u, v) \in E \mid u \in S, v \notin S\}$. This is commonly called the boundary or hair (in [11]) of S . Also, let $\mathcal{O}$ denote the collection of all subsets of V having odd number of vertices with at least three vertices $\mathcal{O} := \{S \subseteq V \mid |S| > 1, |S| \text{ odd}\}$.

The primal objective function is defined as:

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(v)} x_e = 1 \quad \forall v \in V \\ & x_e \in \{0, 1\} \quad \forall e \in E. \end{aligned}$$

This constraint implies that each vertex with a defect is paired with only one edge part of an error chain. The work of Edmonds showed that the above integer linear program can be relaxed to a continuous linear program whose solutions turn out to be integer, provided that we add additional linear constraints involving $\mathcal{O}$ as

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ \text{s.t.} \quad & \sum_{e \in \delta(v)} x_e = 1 \quad \forall v \in V \\ & \sum_{e \in \delta(S)} x_e \geq 1 \quad \forall S \in \mathcal{O} \\ & x_e \geq 0 \quad \forall e \in E. \end{aligned}$$

Now that it is an LP in continuous form, the dual objective can be formed as

$$\begin{aligned} \max_y \quad & \sum_{v \in V} y_v + \sum_{S \in \mathcal{O}} y_S \\ \text{s.t.} \quad & y_u + y_v + \sum_{\substack{S \in \delta(e) \\ S \in \mathcal{O}}} y_S \leq w_e \quad \forall e = (u, v) \in E, \\ & y_S \geq 0, \quad \forall S \in \mathcal{O}. \end{aligned}$$

It is also common to The slack of each edge e is defined as

$$\text{slack}(e) := w_e - (y_v + y_u) - \sum_{\substack{S \in \delta(e) \\ S \in \mathcal{O}}} y_S,$$

and complementary slackness of in linear programming requires that all edges in the final solution are tight, i.e. with zero slack. With these definitions in mind, the Blossom rules for MWPM state:

1. Grow our dual variables y , which define the growth of each region, outward at each step. Initially, only vertex dual variables exist and odd subset variables are added as in step 5.
2. If slack becomes zero for an edge, we consider the following options for the clusters it is incident to:
3. If the dual cluster being touched is unmatched, we match the two defects and terminate growth
4. If the dual cluster being touched already has a valid matching (even parity). We engage in alternating growth between the matched cluster to allow the existing cluster to continue growth through the existing matching. This is to ensure that tight edges remain tight with slack of zero, which effectively means that the closer dual variable to the collision will shrink, and another will grow to permit the other cluster to grow through (see [31]). If alternating growth does not find another matching, another blossom will form around all clusters, with its dual variable.
5. If the dual cluster being touched forms an odd group of clusters, we fuse these into a larger vertex and begin a blossom, which is a new dual variable search region containing the growth of all of the clusters
6. We then grow each blossom outwards until all clusters/blossoms terminate with pair matchings minimizing the edge weight and maximizing our dual variables

It should be noted that this algorithm solves the problem of having an exponential number of dual variables (since we have a dual for each odd subset of vertices), by adding only the nonzero variables when necessary. See the visualization of Figure 10 on an unweighted decoding graph.

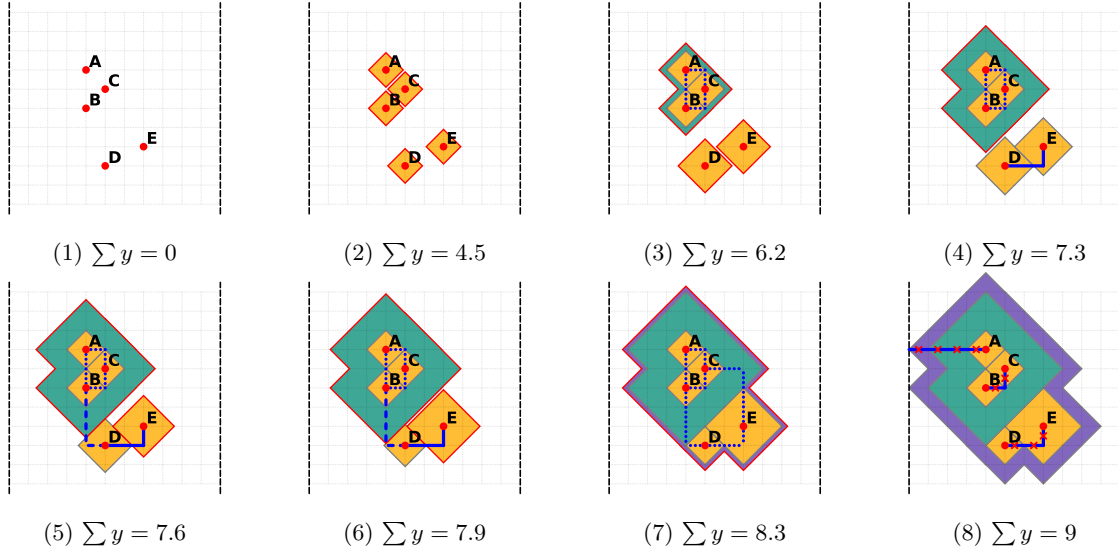


Figure 10: Figure 7, from [33], highlighting the blossom algorithm for an unweighted decoding graph. In the final stage, the nonzero dual variables are $y_A = y_B = y_C = y_D = 1$, $y_E = 2$, $y_{ABC} = 2$, $y_{ABCDE} = 1$. Dual variables corresponding to other odd subsets of vertices (such as y_{BDE}) are zero.

9.1.1 Correlated MWPM

In many CSS codes, including the surface code, the X and Z errors are decoded separately, given the fact that $Y = iXZ$ and that it would register as both an X error and a Z error. However, there is inherent correlation between X and Z errors. Intuitively, if an X error is detected, it could be because of an X or a Y error. As a result, we expect the probability of detecting a Z at that location would increase. Consider a symmetric depolarizing channel

$$\mathcal{E}(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z).$$

Also decompose an error as $E = E_X E_Z$. In the depolarizing channel model, the conditional probabilities are (Equations 1 to 4 of [8])

$$\begin{aligned} \mathbb{P}(E_Z = I \mid E_X = X) &= \frac{1}{2} & \mathbb{P}(E_Z = Z \mid E_X = X) &= \frac{1}{2} \\ \mathbb{P}(E_Z = I \mid E_X = I) &= \frac{1-p}{1-2p/3} & \mathbb{P}(E_Z = Z \mid E_X = I) &= \frac{p/3}{1-2p/3}. \end{aligned}$$

Notice that if we use the conditional probabilities in the definition of w_e , the weight for the Z -component decoding graph will be zero, while other edges will have a constant weight (which will be set to 1 by a constant rescaling). As a result, the authors of [8] propose a decoding method for a CSS code to account for the correlation caused by Y , under the symmetric depolarizing channel:

1. Decode the X -component by MWPM, just as before
2. For any detected X error, set the corresponding edge weight to zero. Likewise, in the syndrome graph for the Z -component decoding, the distance between two syndromes is the same as before but is the number of non-erased edges on the shortest path.
3. Decode the Z component with MWPM on the new graph. This is equivalent to minimizing the weight on non-erased edges.

Figures 11 and 12 illustrate the difference between the uncorrelated and correlated on the toric code. As we can see in the example of Figures 11 and 12, using MPWM without accounting for correlation can cause a logical Z error in the detected outcome.

Note that the above method is only limited to the code-capacity noise model. In the circuit-level noise model, more sophisticated approaches should be used. In Pymatching's [19] implementation of [12], the DEM is first converted to

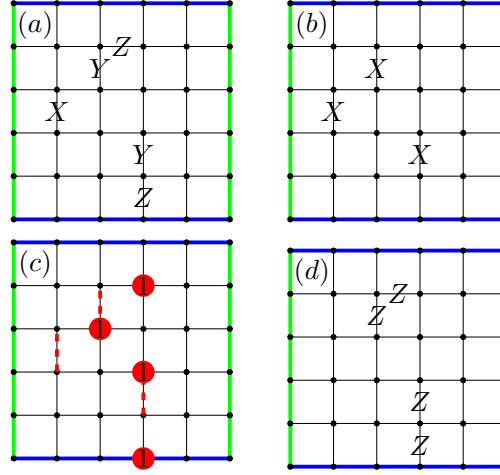


Figure 11: Example that independent MWPM decodes incorrectly, but accounting for correlation can fix (Figure 2 of [8]). (a) The error sequence on a distance-5 toric code. (b) The X component E_X outcome of MWPM. (c) The Z syndromes (dotted red), alongside erased edges (red dashed) created by X errors. (d) Correct MWPM decoding of the z component. The weight is minimized on non-erased edges, while satisfying the syndrome consistency constraint.

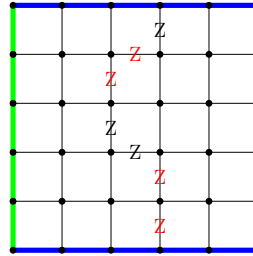


Figure 12: Decoding error that independent MWPM of the Z component gives based on the syndromes of the previous example (black), along with the correct error detection (red). As we can see, these two errors differ by a logical Z , causing a logical error.

a 3D graph to account for noisy measurements as well. Then, a pass of uncorrelated MWPM is run on this graph, with adjustments for the 3D case. Then, similar to the above method, the weights are corrected and another pass is performed.

9.2 Limitations of MWPM and motivation for alternative approaches

We will conclude this section by emphasizing the encountered limitations of MWPM and motivating alternative approaches to decoding. As we have discussed, even the more complex variants of MWPM do not take into account the whole *coset* probability and consider only the most likely error. In addition, they rely on the assumption of independent noise channels. Even when using the DEM to construct the graph like Pymatching [19], some information is lost. It is noteworthy that algorithms such as [13] exist that mix BP with MWPM for better performance, which we will not cover here. Another problem we have not discussed is "leakage"; normally when we implement the qubits on hardware, we use the ground and the first excited states of a particle. A leakage happens when a particle jumps from the intended states $|0\rangle, |1\rangle$ to higher excited states $|2\rangle, |3\rangle$ and so on. [2]

Based on these issues, the motivation for machine learning approaches has been to "let the data speak for itself"; in other words, we should use as little assumptions as possible and use the data directly to train a predictor. As the model becomes more complex, we encounter a natural tradeoff between higher accuracy and longer training and inference costs (explained in the next section). This is even more important given the $1\mu s$ decoding throughput limitation mentioned earlier.

10 Selected neural network architectures

This section concerns standard machine learning concepts as necessary background, and the material can be found in any standard machine learning textbook such as [16]. As a result, to preserve conciseness, unless explicitly indicated otherwise, consider the material to be adapted form [16].

10.1 A short introduction to machine learning: feedforward networks

We start our machine learning background material by introducing feed-forward neural networks. In short, such a network transforms its input x to prediction $\hat{y}$ in the form of

$$\phi_{j+1} = f_j(W_j \phi_j + b_j), \quad \phi_0 = x, \quad \phi_{N+1} = y,$$

where the matrices W_i and biases b_i are trainable, and f_i are nonlinearities (called "activation functions") we can choose prior to training our model. We also define a loss function on predicted and correct predictions to optimize, with a common choice being empirical risk minimization (ERM) as

$$L = \sum_{i=1}^M \ell((x_i, y_i), \hat{y}_i).$$

Then, the parameters are updated sequentially through an optimization algorithm. For the sake of simplicity, we consider the gradient descent updates:

$$(W_i)_{t+1} = (W_i)_t - \eta \nabla_{W_i} L.$$

After the optimization phase, known as "training", the model is ready to predict given unknown data, which is called "inference". It has also been known that a "universal approximation theorem" holds for such networks (see [16] for more details and additional references), which intuitively states that any continuous function can be approximated by such networks, provided that it has sufficient parameters. For more details on this section, see [16].

10.2 Convolutional Neural Networks

While in principle, feedforward neural networks should always perform well, it is usually not the case. In this section, we consider how to build suitable neural networks for images. Since an image and a surface code both have a grid-like structure, our insights on images also hold for surface codes. With that in mind, consider Figure 13, which shows the output of applying a feedforward layer to a matrix, before the nonlinear activation.

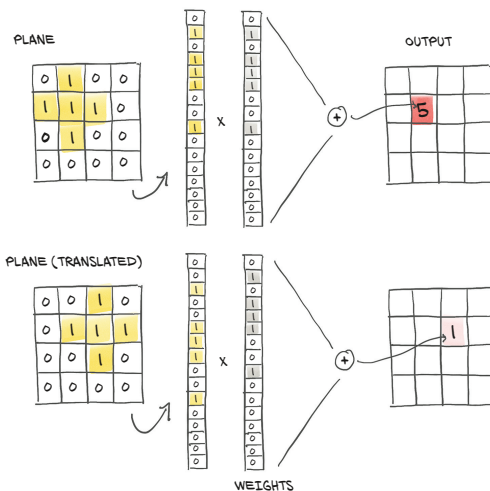


Figure 13: Fully connected network on an image (Figure 7.16 of [24])

In this case, we can see that the layer lacks "translation equivariance"; shifting the image (of a plane, in this case) has changed the shifted output from 5 to 1. This would mean that the filter would have a hard time fitting to a particular pattern. In addition, we will be using more parameters than necessary for our neural network, since we

do not utilize the spatial patterns in the image. This in turn increases training and inference costs, which is more pressing if we recall our latency and throughput constraints for our original goal of QEC decoding.

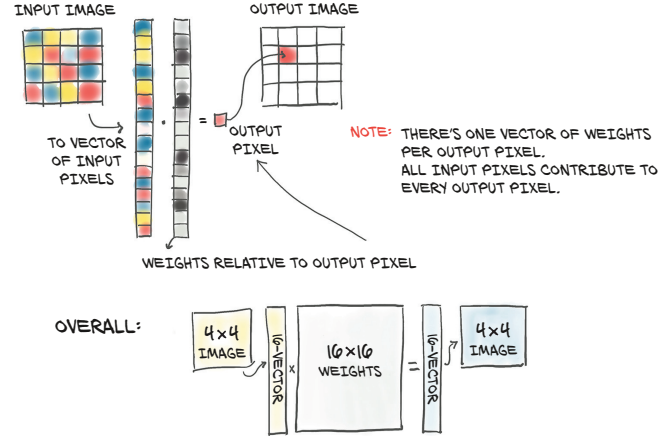


Figure 14: Treating an image like an array of numbers increases the number of required parameters (Figure 7.15 of [24]).

The problems that we discussed above motivate the so-called convolutional neural networks (CNNs). Before we explain how these layers solve the above problems, we will explain equivariance with respect to a group, and then translation equivariance.

Following [4], for linear maps $T_X(g)$ and $T_Y(g)$, we call a function group-equivariant if

$$f(T_X(g)x) = T_Y(g)f(x).$$

For example, if $T_Y = I$ is the identity map and g corresponds to the translation group, the function f is translation invariant. This notion, however, can be extended to more general groups such as rotation and so on.

In the case of one-dimensional inputs, we recall that for the convolution operator $*$ and output $z(t) = h(t) * x(t)$, we have the translation equivariance identity $z(t - \tau) = h(t - \tau) * x(t) = h(t) * x(t - \tau)$. The same identity holds for 2-dimensional convolutions as well, which gives rise to a convolutional layer. A visual demonstration is given in Figure 15, which shows the equivariance and the parameter reduction to only 9 parameters for the whole image.

Note: Since the weights are learnable and "flipping" them has no overall impact, these layers are implemented and viewed as cross-correlations.

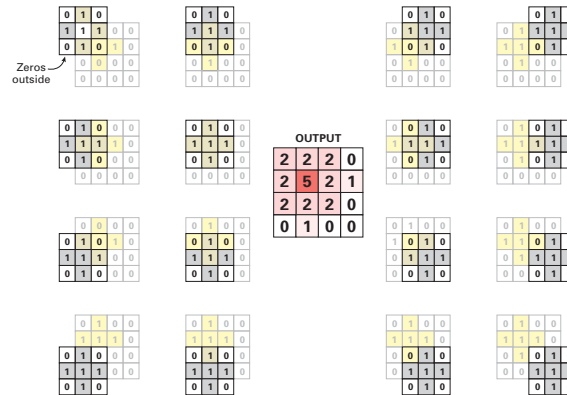


Figure 15: Cross-correlation on an image (Figure 14.7)[16]

Now that we have established convolutional layers, we will briefly examine modified layers that are used in practice. "Strided convolution" happens when the shift (or stride) number is more than one, allowing the filter to parse the image more quickly and reduce parameters. This is particularly useful if the image is very high-resolution

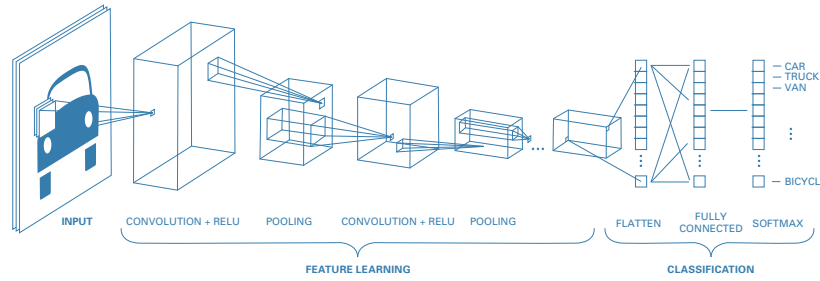


Figure 18: Example of a CNN for image classification (Figure 14.13)[16]

and adjacent points vary slowly. See Figure 16 for an illustration. "Dilated convolution" is also another modified version which increases the "receptive field" of the output without increasing the filter parameters. See Figure 17 for an illustration.

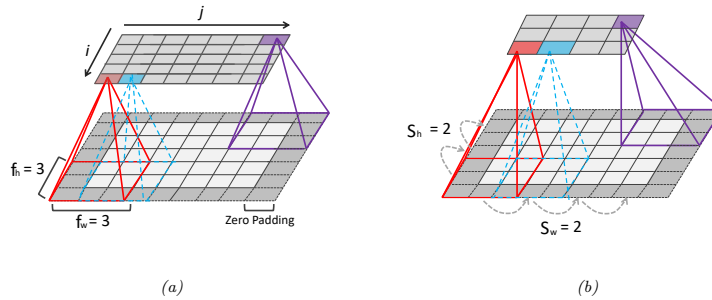
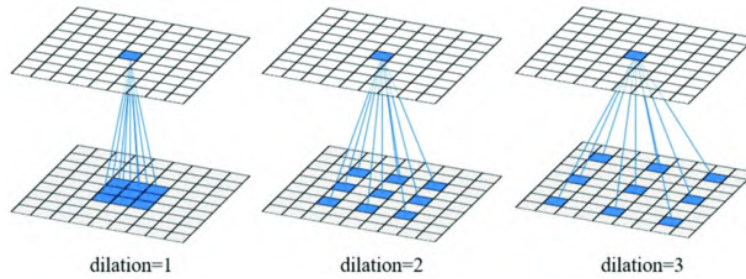


Figure 16: a) Convolution with zero padding. b) Strided Convolution (Figure 14.8)[16]

Figure 17: Dilated convolution for 3×3 filter [5]

A common way to build a practical convolutional neural network is to flatten the features after a number of layers, and use the previously-discussed feedforward layers.

10.3 Recurrent neural networks and transformers

10.3.1 Recurrent neural networks

Now that we are able to process grid-structured data, we need to find a way to process sequential data as well. This is motivated by processing video and text in classic machine learning; in our case, however, it is motivated by processing repeated measurements of stabilizers.

Recurrent neural networks (RNNs) are one of the proposed solution to this challenge; they act as nonlinear state-space models where all the functions are neural networks themselves. Unlike the CNN where we shared parameters through space, we can view this as parameter sharing through time. Several types of RNNs have been included in Figure 19.

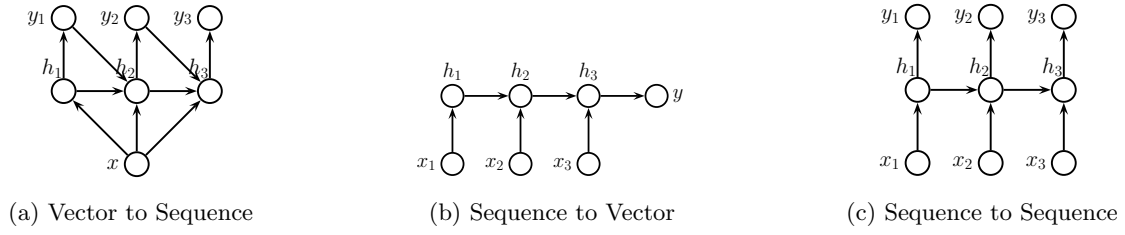


Figure 19: Types of RNN encoding and decoding (Chapter 15)[16]

10.3.2 Attention and transformers

Another way of processing sequential data is through the "attention" mechanism. One way to look at it is through a soft dictionary lookup; we match our inputs against certain keys and determined how aligned they are with each one. Based on that, we choose a probability vector, and the output will be the expected "value", determined from mixing a table of values according to the probability distribution. More concretely, it can be written as (equation 15.34 of [16])

$$\text{Attn}(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) \mathbf{v}_i \in \mathbb{R}^v,$$

where $\mathbf{q}$ is our input query, $(\mathbf{k}_i, \mathbf{v}_i)$ are the key-value pairs, and α is a probability distribution: $0 \leq \alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) \leq 1$ and $\sum_i \alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) = 1$. Often, the attention weights are chosen as a softmax distribution (equation 15.35 of [16]):

$$\alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))}$$

As illustration of this type of attention is provided in Figure 20:

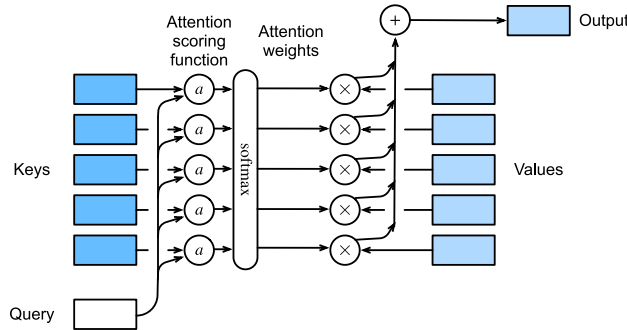


Figure 20: Visual of how attention works (Figure 15.16)[16]

Simplifying even more, a popular choice for attention is

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = S\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V},$$

where $\mathbf{Q} \in \mathbb{R}^{n \times d}$, $\mathbf{K} \in \mathbb{R}^{m \times d}$, $\mathbf{V} \in \mathbb{R}^{m \times v}$ and n is the number of datapoints, d and v are the data and key, and output dimensions, and m is the number of "attention channels". A comparison of an RNN unraveled through the sequence and an RNN with attention on top of the encoder (to be used for decoding) is provided in Figure 21.

Next, we move to the concept of transformers. They were popularized by the "Attention is all you need" [30] paper (260k citations so far) as a way to entirely remove RNNs from sequential processing. We will quickly review their method, as these components have also been used in QEC decoders, as we shall see in later sections.

One key idea of transformers is that we can have a sequence attend to itself; that is, we have the keys, queries and values all come from the input sequence. In addition, we can use multiple parallel "heads", which we can combine so that each head can detect a different pattern in the sequence. More concretely, the multi-head self-attention mechanism is defined as (equation 15.55 of [16] modified for self-attention)

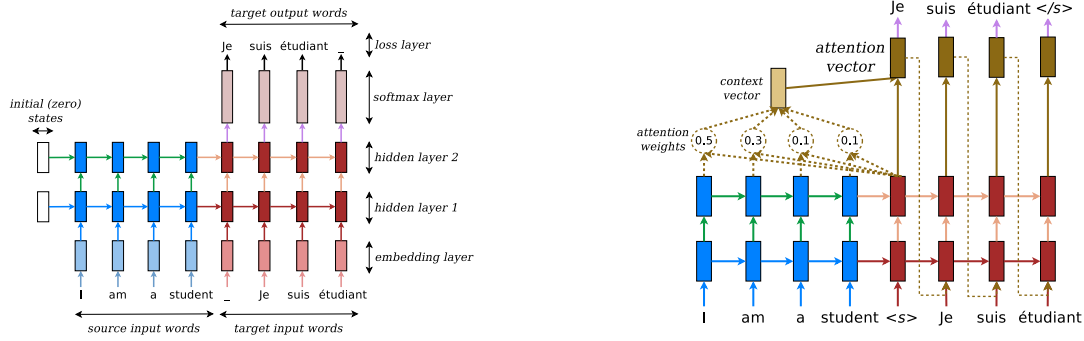


Figure 21: RNN (left) and attention on RNN encoder(right) for a sequence-to-sequence task (Chapter 15) [16]

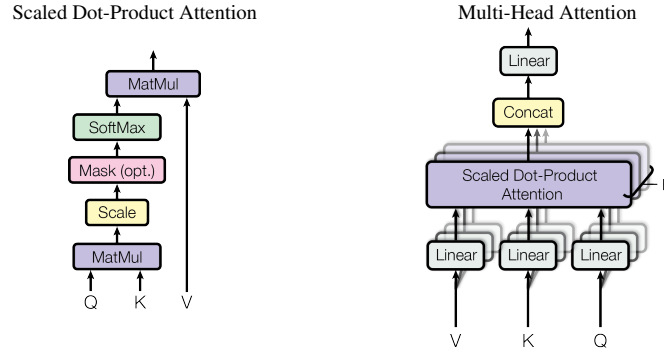


Figure 22: Multi-head attention architecture in transformer [30]

$$\mathbf{h}_i = \text{Attn} \left(\mathbf{W}_i^{(q)} \mathbf{x}, \left\{ \mathbf{W}_i^{(k)} \mathbf{x}, \mathbf{W}_i^{(v)} \mathbf{x} \right\} \right) \in \mathbb{R}^{p_v}$$

where $\mathbf{W}_i^{(q)} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W}_i^{(k)} \in \mathbb{R}^{p_k \times d_k}$, and $\mathbf{W}_i^{(v)} \in \mathbb{R}^{p_v \times d_v}$ are projection matrices. The heads are then concatenated and projected once more to the out space using (equation 15.56 of [16])

$$\mathbf{h} = \text{MHA}(\mathbf{q}, \{\mathbf{k}_j, \mathbf{v}_j\}) = \mathbf{W}_o \begin{pmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{pmatrix} \in \mathbb{R}^{p_o}$$

Similarly, we can have cross-attention when keys and values come from a second sequence $\mathbf{y}$ in the above equations.

10.3.3 Positional embedding

Notice that all the above equations are permutation-invariant. While we want the model to not have a very strict sequential structure, effectively ignoring inputs from the previous and later points, we still need a way to use the position information indirectly. As a result, the so-called positional encodings are used at the beginning of the transformer structure:

- Absolute positional embedding (PE): One common way [30] is to multiply the encoded input sequence by functions

$$p_{i,2j} = \sin \left(\frac{i}{C^{2j/d}} \right), p_{i,2j+1} = \cos \left(\frac{i}{C^{2j/d}} \right)$$

- Rotary positional embedding (RoPE): Encodes relative position in the sequence, by using the rotation matrix:

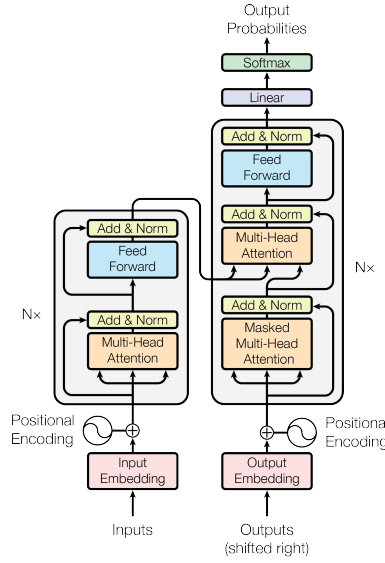


Figure 24: Full transformer architecture (Figure 1 of [30])

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

Figure 23: RoPE Matrix [25]

The authors of [25] show that in the case of linear self-attention, this encoding satisfies

$$\mathbf{q}_m^T \mathbf{k}_n = (\mathbf{W}_q \mathbf{x}_m) \mathbf{R}_{\Theta, m-n}^d (\mathbf{W}_k \mathbf{x}_n),$$

which shows dependence on the relative difference $m - n$. The reader can refer to their paper for the details of their proof. Also, unlike the sine term in absolute positional encoding that vanishes with large position number i , this matrix is orthogonal and thus stable.

10.4 Graph Neural Networks

So far, we have encountered data with clear grid-like spatial structure or sequential data, which also have a clear structure. However, in many cases we will encounter data with irregular structure, best described by a graph. One such case in QEC would be Tanner graphs for qLDPC codes (such as in [17]) other than surface codes. As a result, extending convolutional neural networks to graphs could be a way to keep equivariant layers.

It can be shown that on cyclic graph, the discrete Fourier transform (DFT) is the spectrum of its adjacency and its Laplacian matrix (section 1.3.1 and example 3.5 [18]). To extend this to graph data, we define a "shift operator" $\mathbf{S}$ with eigendecomposition $\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$. Some of the common choices are adjacency and Laplacian matrices. Then, the graph Fourier transform (GFT) of a signal would be $\mathbf{V}^T \mathbf{x}$, and filtering a signal by a spectral filter $\mathbf{h}$ would have the form $\mathbf{V}\mathbf{h}(\mathbf{\Lambda})\mathbf{V}^T \mathbf{x}$. See table 3.3 of [18] for a direct comparison between classic and graph signal processing, including convolutions). As a result, this can be utilized to define graph convolutional layers, as (equations 10 and 13 of [29])

$$\mathbf{X}^{(t+1)} := \rho(H_t(\mathbf{S}) \mathbf{X}^{(t)} + \mathbf{b}^{(t)} \mathbf{1}^\top), \quad H_t(\mathbf{S}) \mathbf{X} := \sum_{i=1}^N \mathbf{v}_i \mathbf{v}_i^\top \mathbf{X} H_t(\lambda_i)^\top,$$

where the filter equations has slightly been changed to accompany multiple graph signals to be filtered.

Since calculating all the graph spectra is not efficient, such filters have been approximated by polynomials over the shift operator $\mathbf{S}$, resulting in graph convolutional networks (GCN), in the form of (equation 16 in [29])

$$\mathbf{X}^{(t+1)} := \rho \left(\sum_{k=0}^K \mathbf{S}^k \mathbf{X}^{(t)} \mathbf{C}_k^\top + \mathbf{b}^{(t)} \mathbf{1}^\top \right).$$

It can be shown (e.g. in [29]) that GCNs are equivalent to message-passing GNNs (MPNNs), written as

$$\mathbf{x}_u^{(t+1)} := \text{UPD}_t \left(\mathbf{x}_u^{(t)}, \text{AGG}_t \left(\{ \mathbf{x}_v^{(t)} \mid v \in N_G(u) \} \right) \right)$$

Each layer aggregates features from local neighborhoods. After L layers, $\mathbf{x}_u^{(L)}$ depends only on the L -hop neighborhood of u .

11 Practical Implementations

11.1 GNN Example

Tanner graphs are a common choice as the underlying graph for GNN layers for QEC decoding. This choice enables comparing graph neural networks to classical algorithms such as BP, as discussed in [17]. To get a more detailed understanding of the parallels, take the logarithm of node values in the BP equations in section 3.3, to reach the equivalent form (equations 3 and 4 of [17])

$$\begin{aligned} \mu_{q \rightarrow c}^{\text{step}:t+1} &= \ell_q + \sum_{c' \in \mathcal{N}(q)/c} \mu_{c' \rightarrow q}^{\text{step}:t} \\ \mu_{c \rightarrow q}^{\text{step}:t+1} &= (-1)^{s_c} 2 \tanh^{-1} \prod_{q' \in \mathcal{N}(c)/q} \tanh \frac{\mu_{q' \rightarrow c}^{\text{step}:t}}{2}, \end{aligned}$$

While another possible approach is to introduce learnable weights in the BP (called neural belief propagation or NBP), equations of the above form (see equations 5 to 7 of [17]), The above reformulation which aggregates nodes additively along with nonlinearities, allows us to use MPNNs to get a more general extension (equation 9 of [17]):

$$\begin{aligned} \mathbf{m}_{i \rightarrow j}^{k-1} &= \text{Message}(\mathbf{h}_i^{k-1}, \mathbf{h}_j^{k-1} \mid \theta^{\text{Message}}) \\ \mathbf{m}_j^{k-1} &= \text{Aggregate}(\{ \mathbf{m}_{i \rightarrow j}^{k-1} \mid i \in \mathcal{N}_j \} \mid \theta^{\text{Aggregate}}) \\ \mathbf{h}_j^k &= \text{Update}(\mathbf{m}_j^{k-1}, \mathbf{h}_j^{k-1} \mid \theta^{\text{Update}}) \\ k &\in \{1, \dots, K\} \end{aligned}$$

An illustration of replacing message-passing in BP with GNN layers is provided in Figure 25. The authors also compare their method to BP, BP with ordered statistics decoding (OSD), and NBP, with results in Figure 26 showing superiority to classical methods.

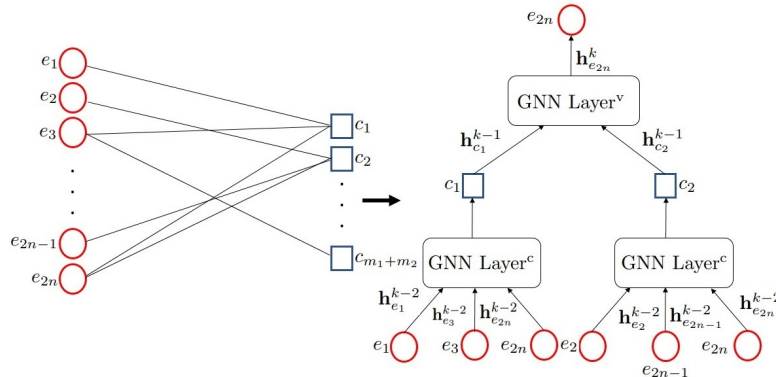


Figure 25: Example structure for decoding qLDPC codes (Figure 1[17])

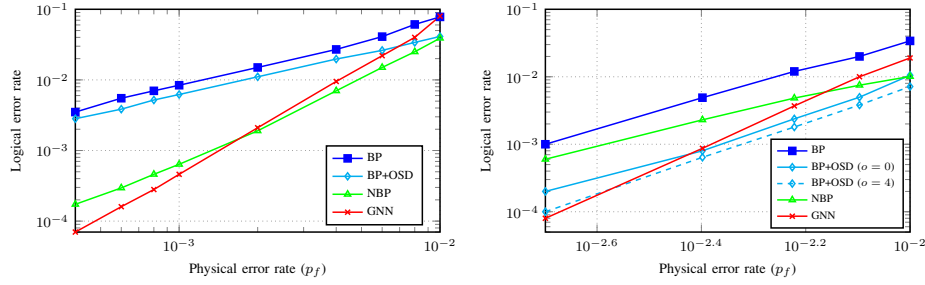


Figure 26: Decoding LER for example of [17] on hypergraph (left) and bicycle (right) codes (Figs. 2, 3[17])

11.2 AlphaQubit 1

AlphaQubit 1 [2], was developed by Google in 2024 for error correction on surface codes specifically, showing that machine learning could effectively be used to handle many of the complexities better than classic methods. We outline some of their main ideas and results before going into more detail:

1. They train the model in two stages: First, they pre-train on synthetic data and fine-tune on scarce experimental data. The synthetic data could be generated from a simple noise model, or could result from creating DEMs out of a subset of experimental data. See Figure 28 for an overview of their overall approach. In addition, they gradually increase the noise parameters during pretraining, as they consider the early stages for the model to learn the decoding task itself.
2. Their complex architecture allows them to feed core complex features. Examples include leakage data, and using the analog outputs of the stabilizer measurements as a proxy to model measurement uncertainty.
3. As we discuss below, their architecture consists of a combination of RNNs for the repeated stabilizer measurements through time, and transformers on top of dilated CNN layers to process the spatial aspect of surface codes. Figure 29 contains an outline of the architecture.
4. Results summary of AlphaQubit 1:
 - They achieve an LER of $(2.901 \pm 0.023) \times 10^{-2}$ at distance 3 and $(2.748 \pm 0.015) \times 10^{-2}$ at distance 5.
 - They also demonstrate better performance than a tensor network, which in their experiments had 3.028 ± 0.023 LER at distance 3 and 2.915 ± 0.01 LER at distance 5.
5. Model Ensembling: In addition, they find that training 20 different models with random initialization and averaging their predictions also improves the decoder performance. It should be noted that such process can be run in parallel and does not affect error correction time.

They used the following circuit noise models to create their pretraining synthetic data:

1. Circuit-related DEMs (up to 2 billion samples):
 - The (even) detection error event correlations p_{ij}
 - A Pauli noise model fitted to device calibration data directly (XEB)
2. Superconducting-inspired 1,000-ns round duration (SI1000): This is a noise model of depolarizing channel with different noise strengths on locations such as 1-qubit and 2-qubit gates, measurements and ancilla resets. See Figure 27 for details. They generated 500 million samples of this model to pretrain.

For finetuning, they partition 325,000 experimental samples with an even index number into training and validation sets. Samples with an odd index are held back as a test set.

Operation	Noise Type	Strength
Z (X) Measurement	Preceded by bitflip (phaseflip) channel	$5p$
Z (X) Reset	Resets to $ 0\rangle$ ($ +\rangle$) and applies a bitflip (phaseflip) channel	$2p$
Resonator Idle	Applies 1q depolarizing channel to qubits not measured/reset	$2p$
2q Clifford	2q depolarizing channel	p
1q Clifford	1q depolarizing channel	$p/10$
Idle	1q depolarizing channel	$p/10$

Figure 27: Noise parameters for the SI1000 model (Extended data Figure 1)

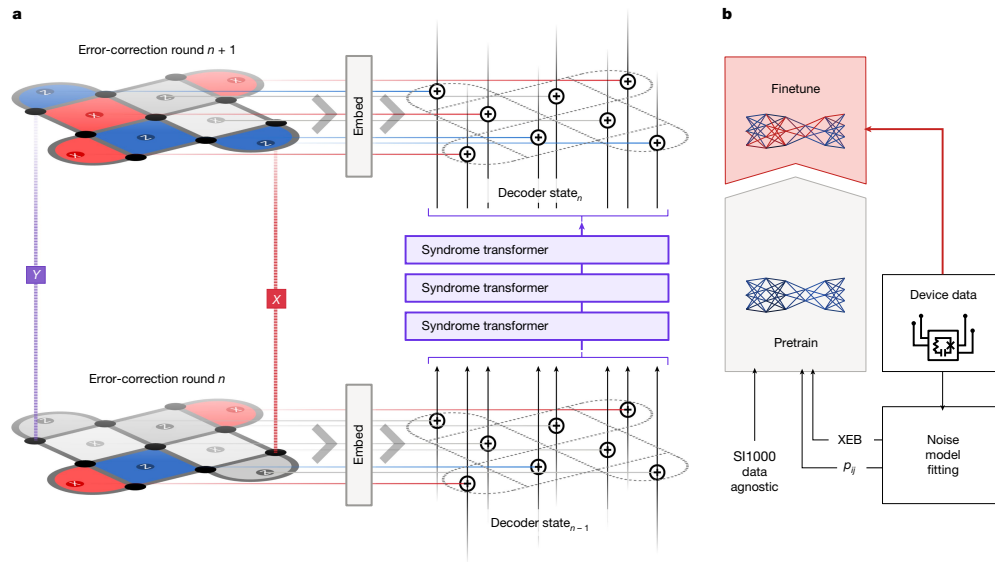


Figure 28: Overview of training method for AlphaQubit 1 (Figure 2)

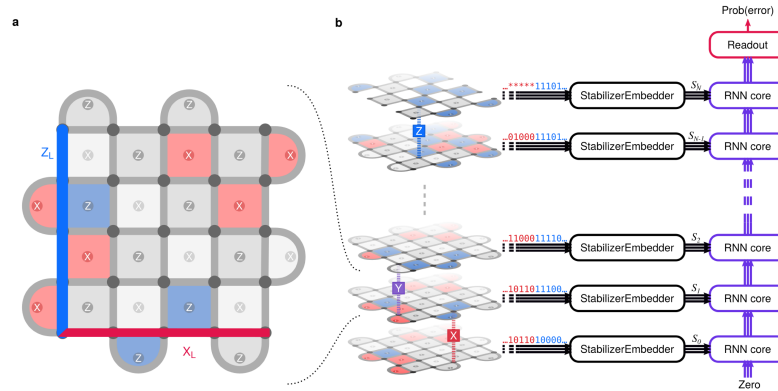


Figure 29: Architecture outline of AlphaQubit 1 (Extended Data Figure 4)

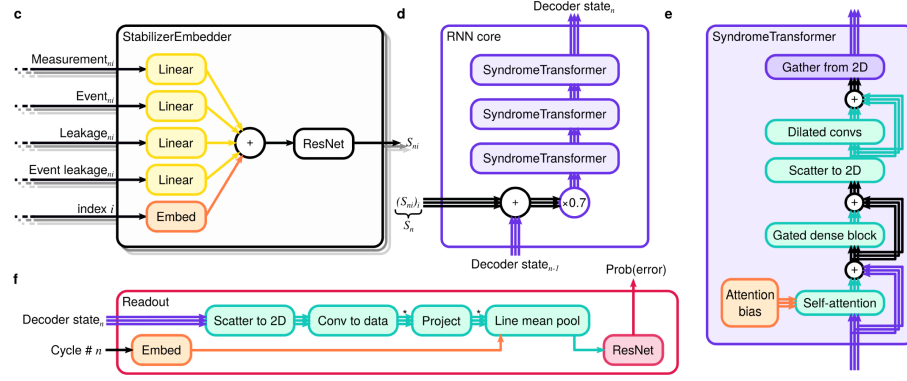


Figure 30: Details of Embedder and RNN core (Extended Data Figure 4)

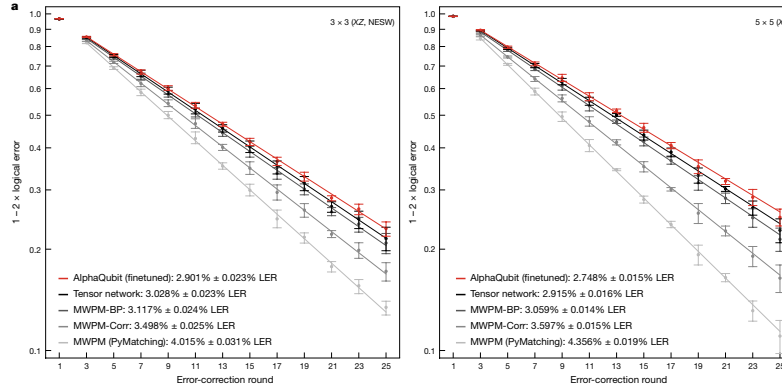
AlphaQubit results: Some of the experiment results are shown in Figures 31, 32. Figure 31 compares the logical error rate after n rounds of measurements, with the metric (Equation 3 of [2])

$$F(n) = 1 - 2E(n) = (1 - p)^n$$

a lower logical error for the fine-tuned AlphaQubit than other methods. In addition, their ordering is intuitive as well: We expect mixing BP with MWPM fare better than correlated MWPM and uncorrelated MWPMs. It is also expected that the tensor network would have a lower error rate than other classic decoding methods.

Figure 32 is important since it shows the importance of fine-tuning as well as the effect of the data used for pretraining. The right-hand figure clearly shows this, with a higher error rate when the simpler SI1000 noise model was used for pretraining, instead of the other DEMs derived from real data. The authors however remark that they are able to reach a better performance than the tensor network method even before fine-tuning with the two realistic DEMs.

Also, the authors report as interesting outcome: While their models were trained for maximum 25 error correction rounds, their models are able to maintain performance even as this number increases significantly on the test set (Figure 34.

Figure 31: Logical error per round on 3×3 and 5×5 Sycamore experiment (Figure 3)

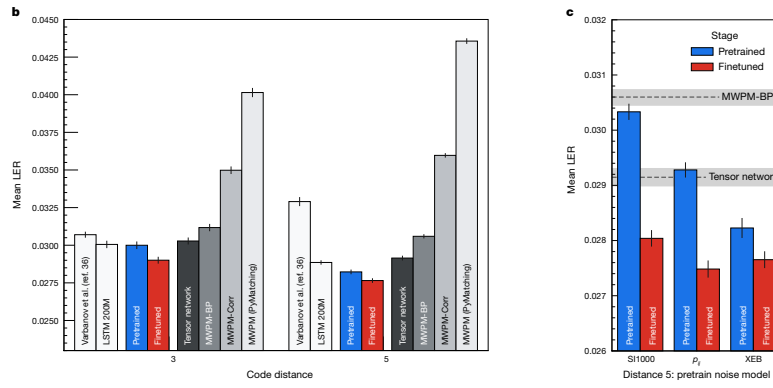


Figure 32: LERs of decoder, before and after fine-tuning on Sycamore experiments (Figure 3)

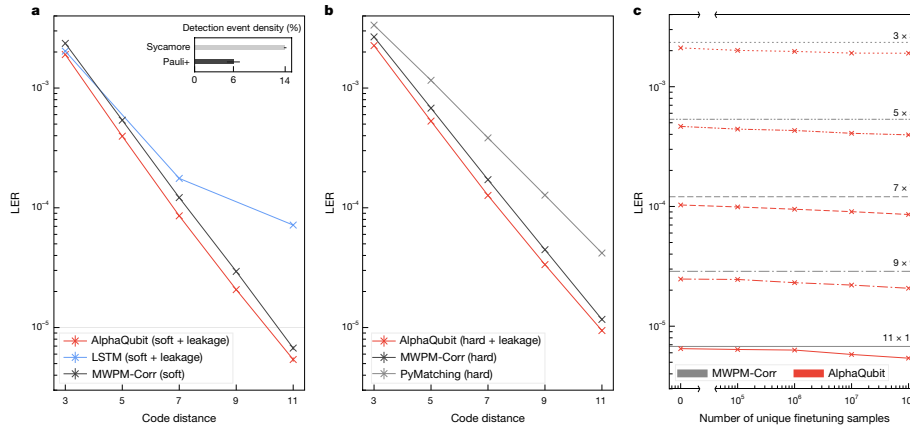


Figure 33: Accuracy tradeoffs: a,b) Effect of increasing code distance. Decoder is finetuned on 100 million samples from Pauli+ after pretraining on (SI1000). c) Accuracy vs. Pauli+ finetuning samples (Figure 4)

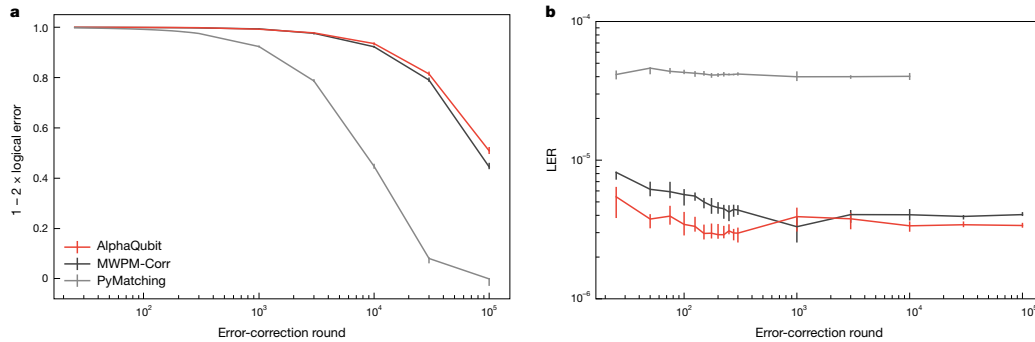


Figure 34: Comparison of AlphaQubit, MWPM-Corr and MWPM (Pymatching-uncorrelated) on large number of error correction rounds. (Figure 4)

Ablation studies: The authors perform tests to measure the effect of simplifying architecture. We provide here a selected list of simplifications, used in their ablation study figures:

- SimpleReadoutStack: Simplify the Readout ResNet.
- SimpleInputStack: Simplify embedding ResNet from 2 layers to 1 layer.
- FewerDims: Reduce the syndrome transformer embedding size
- FewerLayers: Reduce the syndrome transformer depth from 3 to 1

- OnlyEvents: Only provide detection events (remove measurements).
- OnlyMeasurements: Only provide stabilizer measurements (remove events).

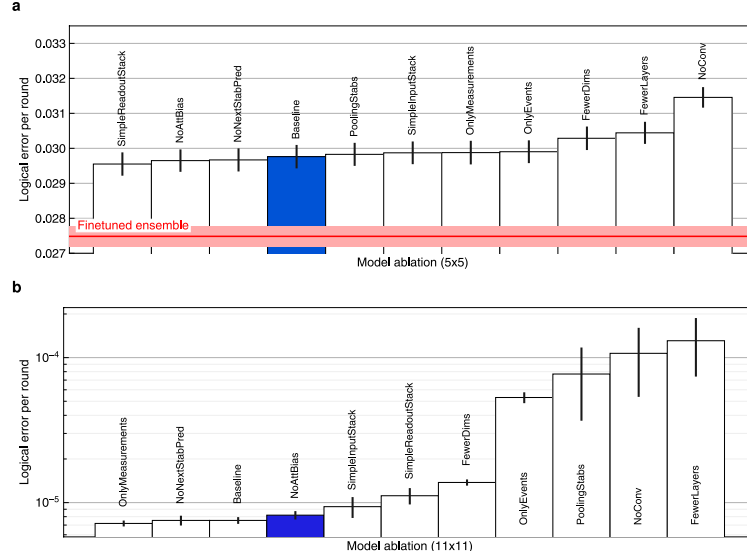


Figure 35: Effect of simplifying decoder structure: a) 5×5 DEM. b) 11×11 Pauli+

Figure 35 indicates that while some design choices such as the bias term inside the attention barely change the performance, other simplifications degrade the LER more heavily. These include having no convolutional layers inside the syndrome transformer block (recall Figure 30), and reducing the number of transformer layers. The "NoConv" degradation matches our intuition that utilizing the grid structure in a surface code is important for achieving good performance. This is even more pronounced for the 11×11 ablation (Figure 35 b), which is also intuitive: For a larger-distance surface code, we expect the grid structure to be much more relevant than for a smaller code. Other aspects like "FewerDims" and "FewerLayers" justify their complex structure as a necessity on top of the CNN layers, with increased degradation on the more complex 11×11 surface code.

11.2.1 Throughput and Latency

Recall that we need a decoding time of $1\mu\text{s}$ or less to deliver a reasonable throughput. The authors of [2] admit they only optimized to the extent of reasonable training times. However, they suggest several steps to reduce the computational load at training and more importantly, inference time. Some of their suggestions include more efficient variants of attention models such as sparse attention, or using "knowledge distillation"; i.e. training a smaller student model based on the less efficient teacher model which has learned the data structure.

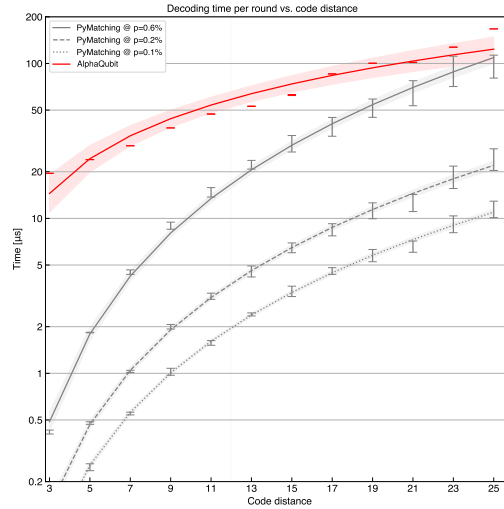


Figure 36: Decoding throughput vs code distance (Extended data Figure 7)

11.3 AlphaQubit 2

AlphaQubit2 [23] was also developed by Google and published in 2026, where they focus on reducing the throughput and decoding latency. In these notes, we focus on the architecture changes from AlphaQubit 1 to AlphaQubit 2; some of the changes are outlined in Figure 37.

1. One of the main directions they go is reducing the amount of recurrence necessary to decode. In AlphaQubit 1, all calculations including the transformer layers were performed inside an RNN which increased decoding time. To alleviate this problem, they first compress every few consecutive measurements and only perform recurrence on the compressed timeframes.
2. In addition, their RNN structure is simplified and is performed before and after transformer layers. This allows some computation be parallelized between different frames and increases speed.
3. No convolutional layers were used in the model. The authors explain that one of their reasons was having a unified architecture for color codes as well, for which the grid structure suitable for CNNs does not hold. Instead, they use the RoPE positional encoding for indicating the 2D structure (which we explained in the background section). More details about the architecture are provided in Figure 38.

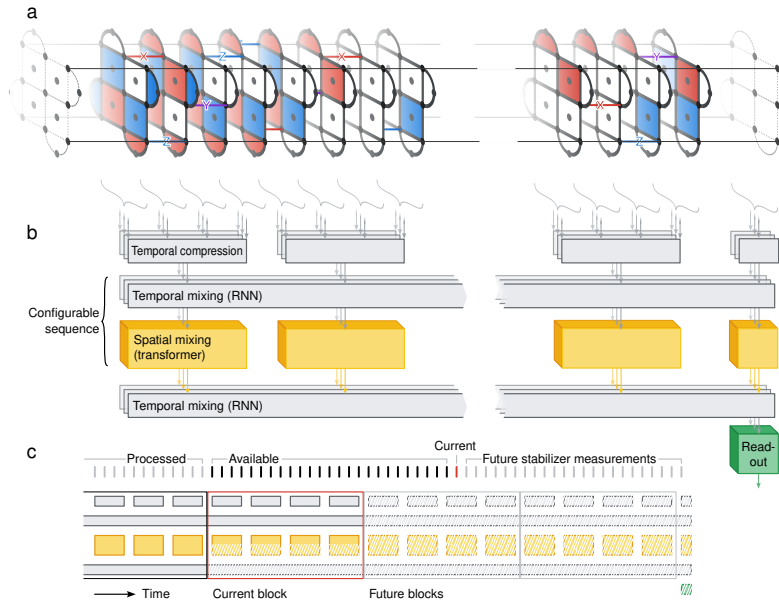


Figure 37: Overview of AlphaQubit 2 for surface-code decoding (Figure 1)

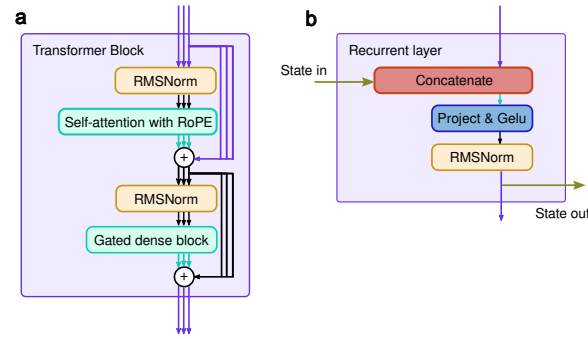
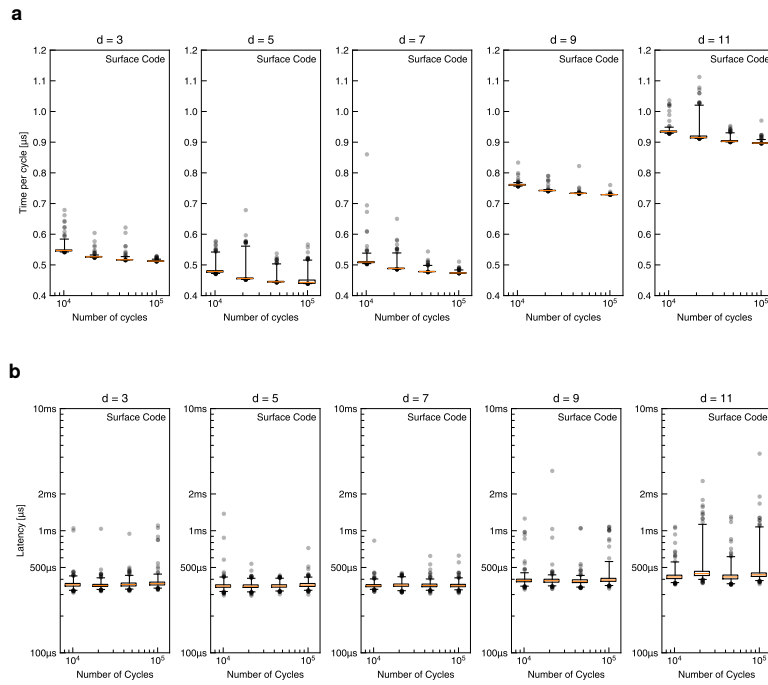
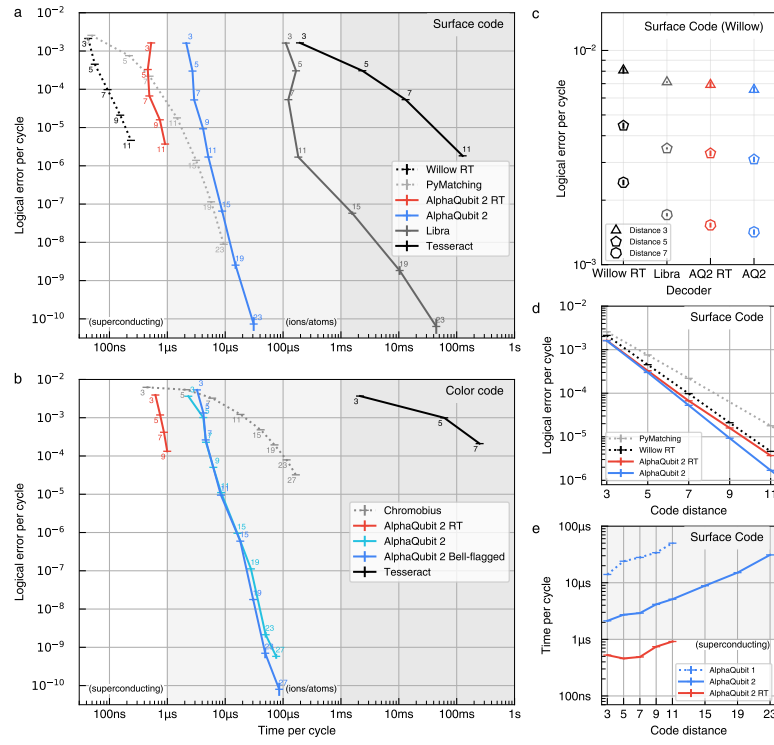


Figure 38: Building blocks in AlphaQubit 2 (Figure S1)

Throughput and latency: A summary of throughput and latency experiments of AlphaQubit 2 is provided in Figures 39 and 40. Notice that the latency measures the time it takes for a single logical qubit to be decoded. Having the appropriate throughput is necessary for every decoder to avoid the backlog problem, which makes the decoder impractical. However, it is not sufficient since the qubit may decay during the processing time. As a result, the latency is also an important decoding metric.

Figure 39 (e) clearly shows the throughput improvement of AlphaQubit 2 over AlphaQubit 1 for every measured code distance; this is further improved for their real-time (RT) model. Other subfigures also show that the real-time model is capable of delivering a lower error rate *and* a lower decoding time than Pymatching. Figure 40 shows the throughput and latency of their method for the surface mode at different distances. We can notice that their throughput is almost always lower than $1\mu s$ but the latency is much higher ($\approx 500\mu s$) which can also be a challenge for reasons mentioned above.



12 Bibliography

References

- [1] F. Battistel et al. *Real-Time Decoding for Fault-Tolerant Quantum Computing: Progress, Challenges and Outlook*. 2023. arXiv: [2303.00054 \[quant-ph\]](#).
- [2] Johannes Bausch et al. “Learning high-accuracy error decoding for quantum processors”. In: *Nature* (Nov. 2024). DOI: [10.1038/s41586-024-08148-8](#).
- [3] Hanyan Cao, Ge Yan, Yuxuan Du, and Feng Pan. *Maximum Likelihood Decoding of Quantum Error Correction Codes*. 2026. arXiv: [2605.17230 \[quant-ph\]](#).
- [4] Taco Cohen and Max Welling. “Group Equivariant Convolutional Networks”. In: *Proceedings of The 33rd International Conference on Machine Learning*. Ed. by Maria Florina Balcan and Kilian Q. Weinberger. Vol. 48. Proceedings of Machine Learning Research. New York, New York, USA: PMLR, 2016, pp. 2990–2999.
- [5] Ximin Cui et al. “Multiscale Spatial-Spectral Convolutional Network with Image-Based Framework for Hyperspectral Imagery Classification”. In: *Remote Sensing* 11.19 (2019). DOI: [10.3390/rs11192220](#).
- [6] Paul L. McFadden David J.C. MacKay Graeme Mitchison. *Sparse Graph Codes for Quantum Error-Correction*. 2004.
- [7] Nicolas Delfosse and Naomi H Nickerson. “Almost-linear time decoding algorithm for topological codes”. In: *Quantum* 5 (2021), p. 595.
- [8] Nicolas Delfosse and Jean-Pierre Tillich. “A decoding algorithm for CSS codes using the X/Z correlations”. In: *2014 IEEE International Symposium on Information Theory*. IEEE, June 2014. DOI: [10.1109/isit.2014.6874997](#).
- [9] Peter-Jan H. S. Derks, Alex Townsend-Teague, Ansgar G. Burchards, and Jens Eisert. *Designing fault-tolerant circuits using detector error models*. 2012. arXiv: [1703.01517 \[quant-ph\]](#).
- [10] Peter-Jan H. S. Derks, Alex Townsend-Teague, Ansgar G. Burchards, and Jens Eisert. *Designing fault-tolerant circuits using detector error models*. 2024. arXiv: [2407.13826 \[quant-ph\]](#).
- [11] Austin G. Fowler. *Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $O(1)$ parallel time*. 2014. arXiv: [1307.1740 \[quant-ph\]](#).
- [12] Austin G. Fowler. *Optimal complexity correction of correlated errors in the surface code*. 2013. arXiv: [1310.0863 \[quant-ph\]](#).
- [13] Oscar Higgott et al. “Improved Decoding of Circuit Noise and Fragile Boundaries of Tailored Surface Codes”. In: *Physical Review X* 13.3 (July 2023). ISSN: 2160-3308. DOI: [10.1103/physrevx.13.031007](#).
- [14] Timo Hillmann et al. *Localized statistics decoding for quantum low-density paritycheck codes*. 2025. arXiv: [2406.18655 \[quant-ph\]](#).
- [15] Vladimir Kolmogorov. “Blossom V: A new implementation of a minimum cost perfect matching algorithm”. In: *Mathematical Programming Computation* (2009). URL: https://www.researchgate.net/publication/225752771-Blossom_V_a_new_implementation_of_a_minimum_cost_perfect_matching_algorithm.
- [16] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. 2022.
- [17] Vukan Ninkovic et al. *Decoding Quantum LDPC Codes Using Graph Neural Networks*. 2024. arXiv: [2408.05170](#).
- [18] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, 2022. ISBN: 9781108428132.
- [19] Craig Gidney Oscar Higgott. *Sparse Blossom: correcting a million errors per core second with minimum-weight matching*. 2025. arXiv: [2303.15933 \[quant-ph\]](#).
- [20] David Poulin and Yeojin Chung. “On the iterative decoding of sparse quantum codes”. In: *Quantum Information and Computation* (2008). arXiv: [0801.1241 \[quant-ph\]](#).
- [21] Joschka Roffe. *LDPC: Python tools for low density parity check codes*. 2022. URL: <https://pypi.org/project/ldpc/>.
- [22] Joschka Roffe, David R. White, Simon Burton, and Earl T. Campbell. “Decoding Across the Quantum LDPC Code Landscape”. In: *Physical Review Research* (2020). arXiv: [2005.07016 \[quant-ph\]](#).
- [23] Andrew W. Senior et al. *A scalable and real-time neural decoder for topological quantum codes*. 2026. arXiv: [2512.07737](#).

- [24] Eli Stevens, Luca Antiga, and Thomas Viehmann. *Deep Learning with PyTorch*. Manning Publications, 2020.
- [25] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: [2104.09864](#).
- [26] R. Tanner. “A recursive approach to low complexity codes”. In: *IEEE Transactions on Information Theory* (1981). URL: <https://ieeexplore.ieee.org/document/1056404>.
- [27] Barbara M. Terhal. *Quantum Error Correction for Quantum Memories*. 2015. arXiv: [1302.3428 \[quant-ph\]](#).
- [28] Simons Institute for the Theory of Computing. *Peeling Algorithms*. URL: <https://www.youtube.com/watch?v=uYAolk0btzo>.
- [29] Antonis Vasileiou et al. *Position: Message-passing and spectral GNNs are two sides of the same coin*. Feb. 2026. DOI: [10.48550/arXiv.2602.10031](#).
- [30] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017.
- [31] Yue Wu. *APS2022 Interpretation of Union Find Decoder on Weighted Graphs and Application to XZZX Surface Code*. URL: <https://www.youtube.com/watch?v=BbhqUHKPdQk>.
- [32] Yue Wu and Lin Zhong. *Fusion Blossom: Fast MWPM Decoders for QEC*. 2023. arXiv: [2305.08307 \[quant-ph\]](#).
- [33] Lin Zhong Yue Wu Namitha Liyanage. *An interpretation of Union-Find Decoder on Weighted Graphs*. 2022. arXiv: [2211.03288 \[quant-ph\]](#).